

Lecture 05 Machine intervals and outward rounding

Prerequisites: Lectures 02 and 04; exact rationals and endpoint interval rules.

Goal: preserve the set guarantee when inputs, operations, and outputs use finite precision.

1. A tiny rounding error can destroy inclusion

0–14 minutes: a failing singleton and the required one-sided contract

Lecture 04 proved formulas over exact real or rational endpoints. A machine implementation must preserve their inequalities. A nearest-rounded approximation can be very accurate and still lie on the wrong side of an endpoint. For a certificate, the direction of the error matters, not just its small magnitude.

Isolate arithmetic rounding by treating two stored binary floats as the exact input data. Their exact rational sum need not equal the nearest-rounded float sum:

```
from fractions import Fraction
x = Fraction.from_float(0.1)
y = Fraction.from_float(0.3)
exact_sum = x + y
rounded_sum = Fraction.from_float(0.1 + 0.3)
assert rounded_sum - exact_sum == Fraction(1, 2**55)
assert not (rounded_sum <= exact_sum <= rounded_sum)
```

The singleton with both endpoints equal to `rounded_sum` misses the exact answer. There is no decimal-input ambiguity in this example: the intended data were the stored floats. If we instead intended $1/10$ and $3/10$, their conversions would be additional operations requiring their own enclosures.

The contract we need. Write $\text{down}(z)$ for rounding toward $-\infty$ and $\text{up}(z)$ for rounding toward $+\infty$. Their defining properties are

$$\text{down}(z) \leq z \leq \text{up}(z).$$

A representable value needs no widening. Otherwise the lower endpoint must lie below the intended value and the upper endpoint above it. Round-to-nearest alone provides neither one-sided promise.

Outward enclosure with four significant binary digits



Accuracy versus inclusion

Why is a zero-width interval at a high-precision approximation generally not an enclosure of $1/10$? Does printing both endpoints as `0.1` prove that the stored endpoints are equal? Explain which objects each statement concerns.

2. Enclose the input before doing arithmetic

14–29 minutes: exact parsing and MPFR conversion in two directions

A safe input pipeline separates two jobs: identify the exact mathematical input, then enclose it in the chosen machine format. For an intended decimal tenth, use an exact rational $1/10$. For an intended stored float, use its exact binary rational value. Neither input interpretation is inherently wrong, but silently switching between them changes the problem.

MPFR supplies arbitrary-precision floating-point arithmetic with specified rounding modes. The Python interface `gmpy2` lets us set precision and rounding in a local context. Here we choose 24 significant binary digits, rather than relying on whatever precision an earlier cell happened to leave active.

```
import gmpy2

def as_fraction(value):
    numerator, denominator = value.as_integer_ratio()
    return Fraction(int(numerator), int(denominator))

intended = gmpy2.mpq(1, 10)
with gmpy2.context(precision=24, round=gmpy2.RoundDown):
    lower = gmpy2.mpfr(intended)
with gmpy2.context(precision=24, round=gmpy2.RoundUp):
    upper = gmpy2.mpfr(intended)

assert as_fraction(lower) <= Fraction(1, 10) <= as_fraction(upper)
print(as_fraction(lower), as_fraction(upper))
```

The audit recovers the exact dyadic values of the finite MPFR endpoints. It does not compare shortened decimal display strings. Both inequalities therefore have an exact rational interpretation.

The order of conversion matters. Passing 0.1 to an arbitrary-precision constructor starts with a Python float whose conversion has already occurred. Extra bits cannot reveal the decimal value the user may have intended. Likewise, rounding an exact rational once to nearest and then using that same result for both endpoints generally loses inclusion at the first step. For a measured interval $[a, b]$, round a down and b up separately.

Negative inputs expose a common mistake. Toward zero is not the same as downward. For a negative nonrepresentable real value, rounding toward zero moves the result upwards. It cannot serve as a guaranteed lower endpoint. For example, a lower enclosure of $-1/10$ must be at least as negative as $-1/10$.

Explain every conversion

Annotate the code with the exact input, the two rounding directions, and the reference used in the assertion. Replace $1/10$ by $-1/10$ and predict the order of the endpoints before executing. Why is the rational reference preferable to an even higher-precision, nearest-rounded approximation for this check?

Reading and notebook. API contracts: [gmpy2 contexts](#) and the [GNU MPFR manual](#), rounding modes.

3. Round the endpoint formulas, in the right direction

29–46 minutes: prove addition and audit all multiplication candidates

Let stored finite endpoints define $X = [a, b]$ and $Y = [c, d]$. Exact addition lies between $a + c$ and $b + d$. Enclosing each extreme outwards therefore gives

$$X + Y \subseteq [\text{down}(a + c), \text{up}(b + d)].$$

The same reasoning gives subtraction bounds $[\text{down}(a - d), \text{up}(b - c)]$. For a denominator excluding zero, reciprocal bounds are $[\text{down}(1/d), \text{up}(1/c)]$. These formulas combine Lecture 04's set argument with the one-sided rounding contract.

For multiplication, let the four exact corner products be $p_1, \dots, p_4$. If $l_i \leq p_i \leq u_i$ for every corner, then

$$\min_i l_i \leq \min_i p_i, \quad \max_i p_i \leq \max_i u_i.$$

Compute all four candidates downward for the lower bound and upward for the upper bound. A nearest-rounded list followed by a change in the rounding mode would not undo the rounding already performed.

```
def multiply_mpfr(X, Y, bits):
    a, b = X
    c, d = Y
    with gmpy2.context(precision=bits, round=gmpy2.RoundDown):
        lower_products = [a*c, a*d, b*c, b*d]
    with gmpy2.context(precision=bits, round=gmpy2.RoundUp):
        upper_products = [a*c, a*d, b*c, b*d]
    for product in lower_products + upper_products:
        if not gmpy2.is_finite(product):
            raise ValueError("A finite enclosure was not obtained")
    return min(lower_products), max(upper_products)

X = (gmpy2.mpfr(-2), gmpy2.mpfr(3))
Y = (gmpy2.mpfr(-4), gmpy2.mpfr(-1))
lo, hi = multiply_mpfr(X, Y, bits=24)
assert as_fraction(lo) <= -12 and as_fraction(hi) >= 8
```

The function assumes ordered MPFR endpoints representing the input sets. Integer endpoints in the demonstration are exact. Nonfinite candidates are rejected; handling infinite endpoints would require a broader arithmetic contract. Results are MPFR endpoints, not Python floats converted from them afterwards.

Check signs and directions independently

For a negative product, should a lower bound be rounded toward zero or toward $-\infty$? Why can the corner attaining the smallest product differ from the corner attaining the largest? Explain why every product needs the appropriate direction.

4. Precision choices and misleading repairs

46–61 minutes: local contexts, genuine uncertainty, and a failed shortcut

More bits can reduce rounding width, but not every wide interval is wide because of rounding. An enclosure of the exact number $1/3$ generally becomes narrower when recomputed at higher precision. The measured interval $[1, 2]$ must still contain every value from one to two. No legitimate precision setting removes that declared uncertainty.

Nor does converting previously rounded endpoints to a wider format recover the original exact input. To refine an input enclosure, return to its exact rational, string, or explicitly specified data set. A finer container can store an old wide interval more accurately without making the interval itself any narrower.

Keep arithmetic settings local. Precision and rounding are part of the meaning of an operation. A notebook should not depend on the order in which unrelated cells happened to run. Context managers make the active settings visible and restore the previous context on exit, including an exceptional exit.

```
before = (gmpy2.get_context().precision, gmpy2.get_context().round)
with gmpy2.context(precision=80, round=gmpy2.RoundUp):
    high = gmpy2.mpfr(gmpy2.mpq(1, 3))
after = (gmpy2.get_context().precision, gmpy2.get_context().round)
assert after == before
assert as_fraction(high) >= Fraction(1, 3)
```

The course's small interval class also rejects mixed-precision interval operands. That policy makes any change of precision explicit; it is not a universal rule that every numerical library must adopt. Agreement between successive precisions is useful evidence, but still does not establish a one-sided bound.

Why not add one ulp at the end? Widening one correctly rounded basic operation by adjacent representable values can be justified under suitable assumptions. It is not a general repair for a whole expression. Consider

```
import math
rounded = (1e16 + 1.0) - 1e16
lo = math.nextafter(rounded, -math.inf)
hi = math.nextafter(rounded, math.inf)
assert not (lo <= 1.0 <= hi)
```

The exact expression on the stored input values equals one. The computed answer is zero, whose neighbouring floats are tiny subnormals. A final one-ulp expansion has no information about the increment lost much earlier in the evaluation.

Where could a guarantee be lost?

Find three distinct points at which inclusion can fail: entering input data, performing an intermediate operation, and exporting the result. Give a remedy for each. If a bound becomes nonfinite, should the program silently substitute a large finite number? What claim would that substitution require?

5. Balls and endpoints: convert the sets, not their printouts

61–77 minutes: midpoint error, radius growth, and an Arb example

A ball $m \pm r$, with $r \geq 0$, represents the real set $[m - r, m + r]$. For exact endpoints $a \leq b$, the formulas $m = (a + b)/2$ and $r = (b - a)/2$ give the same interval. On a machine, rounding m can move the centre. Retaining an unchanged radius may then exclude one of the original endpoints.

If the computed midpoint is $\hat{m} = m + e$ and $|e| \leq \eta$, the triangle inequality gives, for every $x \in [a, b]$,

$$|x - \hat{m}| \leq |x - m| + |e| \leq r + \eta.$$

Thus a radius rounded upwards from $r + \eta$ is safe. Equivalently, choose a radius at least $\max(\hat{m} - a, b - \hat{m})$, evaluating these quantities with appropriate bounds. Midpoint accuracy and radius accuracy have different roles.

Arb provides ball arithmetic through `python-flint`. One useful input construction encloses each exact endpoint in a ball and takes an enclosing union:

```
from flint import arb, ctx, fmpq
with ctx.workprec(80):
    left_ball = arb(fmpq(1, 10))
    right_ball = arb(fmpq(3, 10))
    ball = left_ball.union(right_ball)
    midpoint = ball.mid().fmpq()
    radius = ball.rad().fmpq()
    exact_lower = midpoint - radius
    exact_upper = midpoint + radius
    assert exact_lower <= fmpq(1, 10)
    assert exact_upper >= fmpq(3, 10)
```

Here `fmpq` is an exact rational type. A finite Arb midpoint and radius are dyadic values, so the extracted rational endpoints describe the represented ball exactly. These comparisons avoid an implicit conversion of the rational test value into a wider ball. A containment predicate between two balls answers a different question from membership of one exact rational point.

The input construction may enlarge $[1/10, 3/10]$. If we subsequently evaluate a function on that larger ball, the function must be defined there. An enclosure bridge preserves inclusion; it does not promise identical endpoints or preserve every domain margin automatically.

A ball conversion to audit

Take exact $[a, b] = [1/10, 3/10]$. If the stored midpoint is $1/5 + \eta$ with $\eta > 0$ but the radius remains $1/10$, which endpoint is lost? Show that increasing the radius to $1/10 + \eta$ repairs containment. Why could printing the ball hide this issue?

Reading and notebook. See [python-flint's Arb documentation](#) for the ball API. The course environment uses `gmpy2` 2.3.1 and `python-flint` 0.9.0.

6. Exercises and an endpoint audit

77–90 minutes: one implementation task and one interpretation task

Run the excerpts in page order in an environment with `gmpy2` and `python-flint`; they do not import `vc`. For each claim, keep exact inputs and exact endpoint comparisons separate from approximate decimal displays.

1. **Enclose a rational input.** At 16 and 80 bits, enclose $-1/3$ by converting an exact rational in both directions. Recover the endpoints as `Fraction` values and compare their widths exactly. Explain why a finite binary point interval cannot represent this intended input exactly.
2. **Implement subtraction.** Given ordered finite MPFR intervals $X = [a, b]$ and $Y = [c, d]$, implement a subtraction function using two explicit rounding contexts. Derive its formula first. Test positive, negative, and sign-crossing inputs against exact rational endpoint subtraction.
3. **Audit multiplication.** For $X = [-3, -1]$ and $Y = [-2, 4]$, list the four exact corner products and their extremizers. Check the function on page 3. Then explain why successful tests on these integer endpoints do not establish correct rounding on arbitrary nonrepresentable products.
4. **A misleading export.** Both endpoints of a narrow enclosure of $1/3$ print as `0.333` with three decimal places. If these strings are read back as exact decimals, does the resulting singleton still enclose $1/3$? Describe two safe alternatives for transporting a certificate's endpoints.
5. **A ball invariant.** Suppose $[a, b]$ is exact, $\hat{m}$ is any finite centre, and $\hat{r} \geq \max(\hat{m} - a, b - \hat{m})$. Prove that $[a, b] \subseteq [\hat{m} - \hat{r}, \hat{m} + \hat{r}]$. Which computations need directed rounding if the radius is formed on a machine?

Selected checks. Exercise 2 uses $a - d$ downward and $b - c$ upward. Exercise 3 has products $6, -12, 2, -4$, giving $[-12, 6]$. In Exercise 4 the upper exported endpoint is too small. Exact rational serialization, or decimal endpoints rounded outwards with an explicit digit bound, preserves the required information. For Exercise 5, rearrange each of the two defining radius inequalities.

Exit ticket: what exactly has been trusted?

Identify the real endpoint theorem, the library's rounding contract, the input interpretation, and the final exact comparison in one worked example. Which parts are mathematical reasoning, and which depend on the software implementation? What should be reported if a domain check or finite-endpoint check fails?

Random tests and comparisons against higher precision are valuable diagnostics. They do not prove a universal inclusion contract. The justification combines an all-input mathematical argument with correctly implemented arithmetic operations and explicit conversion rules. Later range calculations will reuse that contract.

Reading and notebook. Companion: `notebooks/05_directed_rounding.ipynb`; Lab 05: outward rounding. Tucker (2011), §2.4, pp. 37–45, especially §2.4.2. The library manuals are linked on pages 2 and 5. Next: dependency, equivalent expressions, and complete domain coverings.